Music Playlist Manager – Sprint 3 Backlog

Software Architecture Design 2026

Corso di Laurea Magistrale in Ingegneria Informatica
Anno Accademico 2025/2026



Docenti: Ritrovato Pierluigi, Apicella Andrea

GitHub: `https://github.com/fra343/Gruppo2-SAD`

Trello: `https://trello.com/b/Mpb9bjYE/sad`

Gruppo 02:

Apicella Francesco	IE22700106
Campagna Lorenzo	IE22700105
Cutolo Raffaele	IE22700113
Gioia Errico	IE22700112

Contents

1 Introduzione **2**
1.1 Obiettivo dello Sprint . . . 2
1.2 Stima della velocity . . . 2
1.3 User Stories selezionate . . . 2

2 Task Breakdown e assegnazioni **3**
2.1 Legenda e scala di stima . . . 3
2.2 US 0 – Setup condiviso di persistenza e discovery (8 ore-uomo totali) . . . 4
2.3 Francesco Apicella (FA): US-17 (5 SP) . . . 4
2.4 Lorenzo Campagna (LC): US-18a + US-18b (3 SP) . . . 4
2.5 Raffaele Cutolo (RC): US-13 + US-14a + US-14b (6 SP) . . . 5
2.6 Errico Gioia (EG): US-15 + US-16 (4 SP) . . . 6

3 Assegnazione delle code review **6**

4 Dettaglio delle User Stories **6**
4.1 US-13: Home page con tracce e playlist più riprodotte *(Assegnato a: Raffaele Cutolo)* . 6
4.2 US-14a: Generazione automatica di playlist per genere *(Assegnato a: Raffaele Cutolo)* . 7
4.3 US-14b: Generazione automatica di playlist per anno *(Assegnato a: Raffaele Cutolo)* . . 7
4.4 US-15: Aggiunta di tag visivi a una traccia *(Assegnato a: Errico Gioia)* . . . 7
4.5 US-16: Generazione automatica di playlist per tag *(Assegnato a: Errico Gioia)* . . . 7
4.6 US-17: Riordino delle tracce in una playlist *(Assegnato a: Francesco Apicella)* [US buffer] 8
4.7 US-18a: Salvataggio di tracce e playlist su disco *(Assegnato a: Lorenzo Campagna)* . . . 8
4.8 US-18b: Caricamento automatico di tracce e playlist all'avvio *(Assegnato a: Lorenzo Campagna)* [US buffer] . . . 8

1 Introduzione

Questa sezione documenta la pianificazione del terzo e ultimo sprint. Lo Sprint 3 costruisce sopra il nucleo consolidato negli sprint precedenti: le entità di dominio e i service introdotti nello Sprint 1 (gestione di tracce e playlist), e il motore di riproduzione, l'aggiornamento della UI in tempo reale e l'infrastruttura di *undo* realizzati nello Sprint 2. Con il nucleo funzionale completato (36 SP su 54), lo Sprint 3 affronta tutte le user stories ancora aperte, chiudendo l'intero scope pianificato del progetto: le funzionalità di arricchimento a Media priorità (home page, generazione automatica di playlist, tag visivi) e quelle accessorie a Bassa priorità (riordino delle tracce e persistenza su disco).

1.1 Obiettivo dello Sprint

Completare le funzionalità residue del Product Backlog: una *home page* che mostra tracce e playlist ordinate per numero di riproduzioni, la *generazione automatica di playlist* a partire da un criterio (genere, anno, tag), l'aggiunta di *tag visivi* (preferito, esplicito, nuova uscita) alle tracce, il *riordino* delle tracce all'interno di una playlist anche durante la riproduzione, e la *persistenza* su disco (salvataggio alla chiusura e caricamento automatico all'avvio).
Al termine dello Sprint l'utente deve poter consultare una home page con le tracce e le playlist più ascoltate, generare automaticamente una playlist filtrando la libreria per genere, anno o tag, applicare e rimuovere tag visivi su una traccia vedendone l'icona ovunque essa compaia, riordinare le tracce di una playlist (anche mentre la riproduzione è in corso) e ritrovare la propria libreria e le proprie playlist al riavvio dell'applicazione. La realizzazione della persistenza comporta inoltre il saldo del *debito di design* sulla serializzazione segnalato nella Review dello Sprint 2 (adattamento del Model alla serializzazione JSON).

1.2 Stima della velocity

Lo Sprint 1 si è chiuso con una **velocity misurata di 12 Story Points** e lo Sprint 2 con una **velocity misurata di 24 Story Points** (24 pianificati, 24 completati). La velocity media sui due sprint è quindi di **18 SP/sprint** e lo scostamento di +6 SP accumulato nello Sprint 1 risulta pienamente riassorbito: con 36 SP completati sui 54 totali, restano **18 SP** che coincidono esattamente con la linea ideale del burndown di progetto.

Nota di pianificazione. Essendo lo Sprint 3 l'ultimo sprint, sono state selezionate **tutte le 18 user stories ancora aperte** per complessivi **18 Story Points**: le cinque a Media priorità (US-13, US-14a, US-14b, US-15, US-16, per 10 SP) e le tre a Bassa priorità (US-17, US-18a, US-18b, per 8 SP). I 18 SP coincidono con la velocity media dimostrata sui due sprint precedenti, rendendo l'obiettivo coerente con la capacità del team. La scelta chiude integralmente lo scope pianificato del backlog (54 SP). Per gestire il rischio, le due user stories a Bassa priorità meno critiche **US-17** (la più ampia, 5 SP) ed **US-18b** (1 SP, dipendente da US-18a) sono marcate come *buffer*: in caso di scostamento a metà sprint sono le prime candidate al rinvio e, coerentemente con il principio di *time-boxing* adottato nel progetto, resterebbero semplicemente nel backlog come obiettivi opzionali, senza compromettere la consegna del prodotto. Si privilegia in ogni caso la qualità del design e del codice rispetto al numero di funzionalità.
La capacity nominale dello Sprint resta di **36 ore-uomo** (9 ore-uomo settimanali per ciascuno dei 4 membri). Coerentemente con la nota di pianificazione, il carico orario complessivo di questo sprint eccede la capacity nominale: l'eccedenza è concentrata sulle US-buffer (US-17 ed US-18b), le prime candidate al rinvio in caso di scostamento. La velocity reale dello Sprint 3 verrà misurata in sede di Sprint Review conclusiva.

1.3 User Stories selezionate

Sono state scelte tutte le user stories ancora aperte dopo lo Sprint 2. Le US-14a, US-14b e US-16 condividono lo stesso schema di *generazione automatica* (filtraggio della `Library` per un criterio e creazione di una nuova `Playlist`), realizzato riusando i metodi di ricerca `findByGenre`, `findByYear` e `findByTag` già previsti nel Model e l'infrastruttura `Command` dello Sprint 2; la US-13 introduce il

conteggio delle riproduzioni e una nuova vista di riepilogo; la US-15 abilita i tag visivi sfruttando l'enum `Tag` (`FAVOURITE`, `EXPLICIT`, `NEW_RELEASE`) già presente nel Model; la US-17 realizza il riordino delle tracce; le US-18a e US-18b implementano la persistenza tramite il pattern DAO (`JsonTrackDAO`, `JsonPlaylistDAO`) già previsto in architettura.

#	User Story	SP
US-13	Come utente voglio visualizzare nella home page le tracce e playlist più riprodotte.	3
US-14a	Come utente voglio generare automaticamente una playlist basata su un genere musicale.	2
US-14b	Come utente voglio generare automaticamente una playlist basata su un anno di pubblicazione.	1
US-15	Come utente voglio aggiungere tag visivi a una traccia (preferito, esplicito, nuova uscita).	2
US-16	Come utente voglio generare automaticamente una playlist basata sui tag delle tracce.	2
US-17	Come utente voglio riordinare le tracce di una playlist (drag & drop o su/giù), anche durante la riproduzione.	5
US-18a	Come utente voglio che le tracce e le playlist vengano salvate su disco alla chiusura dell'applicazione.	2
US-18b	Come utente voglio che le tracce e le playlist vengano ricaricate automaticamente all'avvio.	1
	Totale Story Points	**18**

Table 1: User Stories selezionate per lo Sprint 3.

2 Task Breakdown e assegnazioni

L'organizzazione del lavoro conferma l'approccio a *ownership per user story* adottato negli sprint precedenti: ciascuna US viene presa in carico da un singolo membro che la porta dall'inizio alla fine attraversando tutti i layer architetturali (Model, Service, Controller, View, Test). Le user stories sono state raggruppate per affinità tecnica, così che ogni membro lavori su un'area coerente del prodotto.
Come negli sprint precedenti, alcune componenti di infrastruttura comuni (l'adattamento del Model alla serializzazione, gli scheletri dei DAO concreti, lo scheletro della vista `Home` con il meccanismo di conteggio delle riproduzioni, e il flusso condiviso di generazione automatica delle playlist) vengono definite collettivamente prima che ciascun membro inizi il lavoro autonomo sulla propria US, poiché più user stories vi si appoggiano.
Dipendenza fra US. La US-16 (generazione per tag) si fonda concettualmente sui tag introdotti dalla US-15; la dipendenza è però *debole* a livello di codice, poiché l'enum `Tag` e il metodo `Library.findByTag` esistono già nel Model: la US-16 può essere sviluppata e testata in modo unitario impostando i tag programmaticamente. Analogamente la US-18b (caricamento all'avvio) presuppone la US-18a (salvataggio), poiché condividono i medesimi DAO concreti; per questo sono affidate allo stesso membro.

2.1 Legenda e scala di stima

Legenda: FA = Francesco Apicella LC = Lorenzo Campagna RC = Raffaele Cutolo EG = Errico Gioia.
Le ore di sviluppo sono state stimate con la stessa scala di riferimento interna degli sprint precedenti: **1 SP = 2.5h**, **2 SP = 4h**, **3 SP = 6h**, **5 SP = 9h**. La progressione non è perfettamente lineare per riflettere la maggiore incertezza delle US più ampie. Coerentemente con la nota di pianificazione, il carico orario complessivo di questo sprint eccede la capacity nominale: l'eccedenza è concentrata sulle US-buffer (US-17 ed US-18b), le prime candidate al rinvio in caso di scostamento.

2.2 US 0 – Setup condiviso di persistenza e discovery (8 ore-uomo totali)

Lavoro preparatorio distribuito tra i membri prima dello split sulle singole US:

- Adattamento del Model alla serializzazione JSON: costruttori e annotazioni Jackson su `Track` e `Playlist`, gestione della serializzazione di `ObservableList` (lista interna serializzabile esposta come vista osservabile) – saldo del debito di design segnalato nella Review dello Sprint 2 (FA + LC, 2h ore-uomo);
- Scheletri dei DAO concreti `JsonTrackDAO` e `JsonPlaylistDAO` con le firme `save`/`load` e il mapping verso il formato JSON (LC, 1h);
- Scheletro della vista `Home.fxml` e del controller `HomeController`, con il meccanismo di conteggio delle riproduzioni (`playCount`) nel `PlaybackService` (RC, 1h);
- Definizione del flusso comune di *generazione automatica* (interrogazione del metodo di ricerca della `Library` e creazione della `Playlist` riusando `CreatePlaylistCommand` e `AddTrackToPlaylistCommand`), verifica di `findByGenre`/`findByYear`/`findByTag` e predisposizione del set di icone per i tag (EG, 1h);
- Sessione di design collettiva: formato JSON e contratti DAO, semantica del conteggio riproduzioni, flusso di generazione, resa grafica dei tag (tutti, 0.75h ciascuno = 3h ore-uomo).

2.3 Francesco Apicella (FA): US-17 (5 SP)

Setup condiviso (2h).

- Adattamento del Model alla serializzazione JSON, in coppia con LC (2h).

US-17 – Riordino delle tracce in una playlist (5 SP, 9h). ***[US buffer]***

- Riordino delle tracce nella `ListView` della playlist tramite drag & drop (o pulsanti su/giù), con aggiornamento immediato dell'ordine interno degli indici di `Playlist.tracks` (3.5h);
- Garanzia del corretto comportamento durante la riproduzione: la traccia corrente non subisce blocchi e l'ordine futuro di riproduzione rispetta la nuova sequenza, sfruttando il riferimento condiviso alla stessa `Playlist` nel `PlaybackService` (3h);
- Test JUnit sul riordino (nuovo ordine riflesso negli indici) e sul riordino durante la riproduzione (continuità della traccia corrente) (2.5h).

Rifinitura e code review (1h).

- Javadoc sulla logica di riordino + test sul codice US-17 (0.5h);
- Code review di US-13, US-14a e US-14b (RC) (0.5h).

2.4 Lorenzo Campagna (LC): US-18a + US-18b (3 SP)

Setup condiviso (3h).

- Adattamento del Model alla serializzazione JSON, in coppia con FA (2h);
- Scheletri dei DAO concreti `JsonTrackDAO` e `JsonPlaylistDAO` (1h).

US-18a – Salvataggio di tracce e playlist su disco (2 SP, 4h).

- Implementazione di `JsonTrackDAO.save` e `JsonPlaylistDAO.save` (serializzazione Jackson della libreria e delle playlist su file JSON) (2h);
- Trigger del salvataggio alla chiusura dell'applicazione, agganciato all'evento di chiusura dello stage (1h);

- Test JUnit sulla serializzazione, con test di persistenza `save` → `load` che verifica l'integrità dei dati (1h).

US-18b – Caricamento automatico all'avvio (1 SP, 2.5h). ***[US buffer]***

- Implementazione di `load()` nei DAO e collegamento all'avvio per ricostruire libreria e playlist; gestione del file assente o vuoto alla prima esecuzione (1.5h);
- Test JUnit sul caricamento e sul caso di file assente (1h).

Rifinitura e code review (1h).

- Javadoc sui DAO concreti e sui flussi di salvataggio/caricamento + test sul codice US-18a e US-18b (0.5h);
- Code review di US-17 (FA) (0.5h).

2.5 Raffaele Cutolo (RC): US-13 + US-14a + US-14b (6 SP)

Setup condiviso (2h).

- Scheletro di `Home.fxml`, `HomeController` e meccanismo di conteggio riproduzioni nel `PlaybackService` (1h);
- Partecipazione alla sessione di design collettiva (1h).

US-13 – Home page con tracce e playlist più riprodotte (3 SP, 6h).

- Esposizione del conteggio riproduzioni a livello di traccia e di playlist e logica di ordinamento decrescente per numero di riproduzioni (2.5h);
- Completamento di `Home.fxml` e binding in `HomeController` delle liste ordinate, con aggiornamento automatico al variare dei conteggi (2h);
- Test JUnit sull'ordinamento e sul caso di assenza di riproduzioni (1.5h).

US-14a – Generazione automatica per genere (2 SP, 4h).

- Selettore del genere e invocazione di `Library.findByGenre`; creazione della nuova playlist con le tracce corrispondenti riusando i Command esistenti; gestione del caso "nessuna traccia del genere selezionato" (2.5h);
- Test JUnit per entrambi gli scenari (genere con tracce; genere senza tracce) (1.5h).

US-14b – Generazione automatica per anno (1 SP, 2.5h).

- Selettore dell'anno e invocazione di `Library.findByYear`, riusando il flusso di generazione della US-14a (1.5h);
- Test JUnit per gli scenari con e senza tracce dell'anno selezionato (1h).

Rifinitura e code review (1h).

- Javadoc su `HomeController` e sul flusso di generazione per genere/anno + test sul codice US-13, US-14a e US-14b (0.5h);
- Code review di US-15 e US-16 (EG) (0.5h).

2.6 Errico Gioia (EG): US-15 + US-16 (4 SP)

Setup condiviso (1h).

- Verifica dei metodi di ricerca della `Library` (in particolare `findByTag`), predisposizione delle icone dei tag e partecipazione alla sessione di design collettiva (1h).

US-15 – Aggiunta di tag visivi a una traccia (2 SP, 4h).

- Logica di applicazione/rimozione di un tag (`FAVOURITE`, `EXPLICIT`, `NEW_RELEASE`) sull'insieme `tags` della `Track` (2h);
- Resa grafica dell'icona del tag accanto alla traccia, propagata a tutte le viste in cui la traccia compare (libreria e playlist) tramite il binding alle property del Model (1h);
- Test JUnit sull'aggiunta e rimozione dei tag e sulla loro corretta esposizione (1h).

US-16 – Generazione automatica per tag (2 SP, 4h).

- Invocazione di `Library.findByTag` per il tag selezionato e creazione di una playlist (es. "I miei Preferiti" per il tag `FAVOURITE`) riusando il flusso di generazione condiviso e i Command esistenti (2.5h);
- Comando di interfaccia per richiedere la generazione della playlist per tag (0.5h);
- Test JUnit sulla generazione (set di tracce impostato programmaticamente, indipendente dalla UI della US-15) (1h).

Rifinitura e code review (1h).

- Javadoc sulla logica dei tag e sul flusso di generazione per tag + test sul codice US-15 e US-16 (0.5h);
- Code review di US-18a e US-18b (LC) (0.5h).

3 Assegnazione delle code review

Si conferma la pratica delle code review incrociate degli sprint precedenti, con un nuovo accoppiamento ruotato per allargare ulteriormente la conoscenza incrociata del codice fra i membri del team. A differenza dello Sprint 2 (coppie reciproche), lo Sprint 3 adotta una *rotazione ciclica* (FA $\rightarrow$ RC $\rightarrow$ EG $\rightarrow$ LC $\rightarrow$ FA): ogni membro rivede il lavoro di un altro senza reciprocità diretta.

Owner della US	**Reviewer**
FA (US-17)	LC
LC (US-18a, US-18b)	EG
RC (US-13, US-14a, US-14b)	FA
EG (US-15, US-16)	RC

Table 2: Assegnazione delle code review fra i membri del team per lo Sprint 3.

4 Dettaglio delle User Stories

4.1 US-13: Home page con tracce e playlist più riprodotte *(Assegnato a: Raffaele Cutolo)*

- Implementare il conteggio delle riproduzioni a livello di traccia e di playlist, alimentato dal `PlaybackService` a ogni avvio di riproduzione.

- Completare la vista `Home.fxml` e implementare in `HomeController` l'ordinamento decrescente per numero di riproduzioni, con aggiornamento automatico tramite binding.
- Scrivere i test JUnit sull'ordinamento dei risultati e sul comportamento in assenza di riproduzioni.
- Redigere la Javadoc per `HomeController` e per la logica di conteggio e ordinamento.

4.2 US-14a: Generazione automatica di playlist per genere *(Assegnato a: Raffaele Cutolo)*

- Implementare il selettore del genere e l'invocazione di `Library.findByGenre`, creando una nuova playlist con le tracce corrispondenti tramite i Command esistenti.
- Gestire il caso di assenza di tracce per il genere selezionato mostrando un messaggio informativo, senza creare alcuna playlist.
- Scrivere i test JUnit per entrambi gli scenari (genere con tracce corrispondenti; genere senza tracce).
- Redigere la Javadoc per il flusso di generazione per genere.

4.3 US-14b: Generazione automatica di playlist per anno *(Assegnato a: Raffaele Cutolo)*

- Implementare il selettore dell'anno e l'invocazione di `Library.findByYear`, riusando il medesimo flusso di generazione della US-14a.
- Gestire il caso di assenza di tracce per l'anno selezionato con messaggio informativo e nessuna creazione di playlist.
- Scrivere i test JUnit per gli scenari con e senza tracce dell'anno selezionato.
- Redigere la Javadoc per il flusso di generazione per anno.

4.4 US-15: Aggiunta di tag visivi a una traccia *(Assegnato a: Errico Gioia)*

- Implementare la logica di aggiunta e rimozione di un tag visivo (`FAVOURITE`, `EXPLICIT`, `NEW_RELEASE`) sull'insieme `tags` della `Track`.
- Realizzare la resa grafica dell'icona del tag accanto alla traccia, propagata a tutte le viste in cui la traccia compare (libreria e playlist) tramite il binding alle property del Model.
- Scrivere i test JUnit sull'aggiunta e rimozione dei tag e sulla loro corretta esposizione.
- Redigere la Javadoc per la logica dei tag e per la resa dell'icona.

4.5 US-16: Generazione automatica di playlist per tag *(Assegnato a: Errico Gioia)*

- Implementare l'invocazione di `Library.findByTag` per il tag selezionato e la creazione di una playlist (ad esempio "I miei Preferiti" per il tag `FAVOURITE`), riusando il flusso di generazione condiviso e i Command esistenti.
- Realizzare il comando di interfaccia per richiedere la generazione della playlist per tag.
- Scrivere i test JUnit sulla generazione a partire da un tag, impostando le tracce associate in modo programmatico (indipendentemente dalla UI della US-15).
- Redigere la Javadoc per il flusso di generazione per tag.

4.6 US-17: Riordino delle tracce in una playlist *(Assegnato a: Francesco Apicella)* [US buffer]

- Implementare il riordino delle tracce nella `ListView` della playlist (drag & drop o pulsanti su/giù), aggiornando immediatamente l'ordine interno degli indici di `Playlist.tracks`.
- Garantire che, se la playlist è in riproduzione, la traccia corrente non subisca blocchi e che l'ordine futuro di riproduzione rispetti la nuova sequenza impostata dall'utente.
- Scrivere i test JUnit sul riordino (nuovo ordine riflesso negli indici) e sul riordino durante la riproduzione.
- Redigere la Javadoc per la logica di riordino.

4.7 US-18a: Salvataggio di tracce e playlist su disco *(Assegnato a: Lorenzo Campagna)*

- Implementare `JsonTrackDAO.save` e `JsonPlaylistDAO.save`, serializzando libreria e playlist su file JSON tramite Jackson.
- Agganciare il salvataggio all'evento di chiusura dell'applicazione, in modo che i dati siano persistiti integri al termine della sessione.
- Scrivere i test JUnit sulla serializzazione (test di persistenza `save`/`load` con dati integri).
- Redigere la Javadoc per i DAO concreti e per il flusso di salvataggio.

4.8 US-18b: Caricamento automatico di tracce e playlist all'avvio *(Assegnato a: Lorenzo Campagna)* [US buffer]

- Implementare `load()` nei DAO concreti e collegarlo all'avvio dell'applicazione per ricostruire libreria e playlist dalla sessione precedente.
- Gestire il caso di file assente o vuoto alla prima esecuzione, avviando l'applicazione con uno stato vuoto e senza errori.
- Scrivere i test JUnit sul caricamento e sul caso di file assente.
- Redigere la Javadoc per il flusso di caricamento all'avvio.